

AI Specialization 2: Resolution, Automatic Proving, and Model Search

AI Specialization 2: LUP

Resolution in first-order logic, automatic proving, Boolean and predicate proving, and model search in generic domains.

Sources used: official CTU CourseWare page for BE4M36LUP, which lists lectures on propositional logic, SAT solving, Prolog, Herbrand models, SLD resolution, FOL, model finding, equality, paramodulation, subsumption, and superposition; CTU state-exam topic list; standard logic programming and automated-reasoning material.

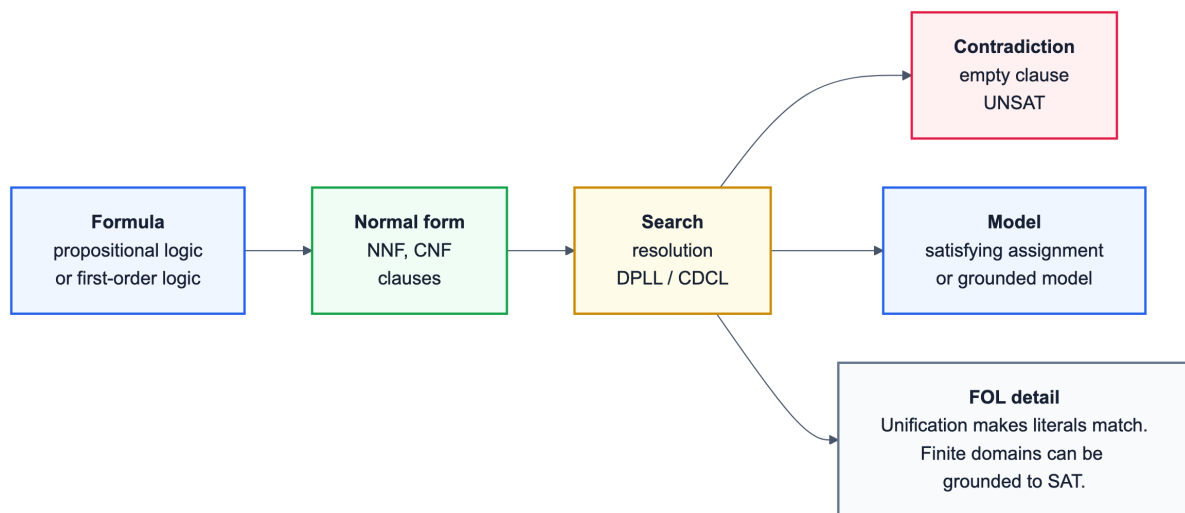


Figure 1: Automatic proving pipeline: formulas are normalized into clauses, then searched for a model or contradiction.

LUP in Plain English

LUP is about proving by search. Convert formulas into a form a machine can search, then either find a model or derive a contradiction.

1. Normalize.

CNF, clauses, Skolem form.

2. Search.

DPLL/CDCL, SLD, resolution, saturation.

3. Interpret result.

Model found, unsatisfiable, or proof by contradiction.

SAT and CNF

Boolean proving becomes clause search.

- CNF = conjunction of clauses.
- Tseitin avoids exponential CNF blowup.
- Resolution derives new clauses until contradiction.

DPLL/CDCL

Modern SAT solving is smart backtracking.

- Unit propagation forces assignments.
- Backtracking explores choices.
- Clause learning records why conflict happened.

Prolog and SLD

Logic programs answer goals by resolution.

- Horn clauses have at most one positive literal.
- SLD resolves selected goal with program rules.
- Negation as failure means "not provable".

First-order resolution

Variables require unification and Skolemization.

- Skolemization removes existential variables.
- Unification finds substitutions that make terms equal.
- Saturation keeps deriving clauses until done or contradiction.

Equality and models

Equality needs special handling; model search grounds domains.

- Paramodulation/superposition reason with equality.
- Grounding finite domains reduces to SAT.
- Subsumption removes redundant clauses.

How to answer

Say what representation the solver searches.

- Formula to clauses.
- Rules for deriving consequences.
- Stopping condition: model or empty clause.

Exam sentence: Resolution proves unsatisfiability by deriving the empty clause; in first-order logic this needs unification so clauses can match after substitution.

Three traps

- CNF conversion and Tseitin encoding are not the same thing.
- Skolemization preserves satisfiability, not logical equivalence.
- Negation as failure is procedural, not classical negation.

1. Propositional Normal Forms and SAT

Normal forms

Propositional formulas are built from variables, connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$.

Negation normal form (NNF): negations occur only in front of variables. Convert by eliminating $\Rightarrow, \Leftrightarrow$ and pushing negations inward with De Morgan rules.

Conjunctive normal form (CNF):

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each clause C_i is a disjunction of literals.

Disjunctive normal form (DNF): disjunction of conjunctions of literals.

For SAT solving, CNF is central. Naive distributive CNF conversion may blow up exponentially; Tseitin transformation introduces auxiliary variables and produces an equisatisfiable CNF of linear size.

SAT problem

SAT asks whether a propositional formula has a satisfying truth assignment. CNF-SAT is NP-complete. A clause is satisfied if at least one of its literals is true.

Resolution rule

For clauses:

$$(A \vee x), \quad (B \vee \neg x)$$

derive resolvent:

$$A \vee B.$$

Resolution is sound: every assignment satisfying the premises satisfies the resolvent. It is refutation-complete for propositional logic: a CNF is unsatisfiable iff repeated resolution can derive the empty clause $\square$.

To prove entailment $\Gamma \models \phi$, show $\Gamma \wedge \neg\phi$ is unsatisfiable.

Simple explanation

Resolution is a proof by contradiction. Add the negation of what you want to prove, then simplify clauses until contradiction appears. The empty clause means “there is no way to make all of this true at once.”

Example

From clauses $(P \vee Q)$ and $(\neg P \vee R)$, resolve on P to derive $(Q \vee R)$. If later you also have $(\neg Q)$ and $(\neg R)$, repeated resolution can derive the empty clause, proving inconsistency.

Unit propagation

A unit clause contains one literal, e.g. (x) . If a CNF contains (x) :

- clauses containing x are satisfied and removed,
- literal $\neg x$ is removed from all remaining clauses.

If an empty clause appears, conflict is detected. Unit propagation is the core inference step in DPLL/CDCL SAT solvers.

DPLL

DPLL is recursive backtracking with propagation:

1. Apply unit propagation.
2. Apply pure literal elimination if desired.
3. If all clauses satisfied, return SAT.
4. If empty clause, return UNSAT.
5. Pick unassigned variable and branch on true/false.

Clause learning and CDCL

Conflict-Driven Clause Learning (CDCL) improves DPLL:

- maintain implication graph from decisions and unit propagations,
- when conflict occurs, analyze it,
- derive a learned clause preventing the same conflict,
- backjump non-chronologically,
- use heuristics such as VSIDS and restarts.

Learned clauses are logical consequences of the formula, usually derived by resolution.

SMT and lazy approach

Satisfiability Modulo Theories (SMT) extends SAT with background theories, e.g. linear arithmetic, arrays, bit-vectors, equality with uninterpreted functions.

Lazy SMT architecture:

1. Replace theory atoms by Boolean variables.
2. SAT solver proposes a Boolean assignment.
3. Theory solver checks whether selected theory literals are consistent.
4. If inconsistent, theory solver returns a conflict clause.
5. SAT solver learns it and continues.

This combines efficient Boolean search with specialized theory reasoning.

Simple explanation

SAT handles the yes/no structure, while the theory solver handles meanings such as arithmetic or arrays. The SAT solver may suggest a Boolean combination that looks possible, and the theory solver says whether it is actually possible in the intended theory.

2. Logic Programming and Prolog

Definite and Horn clauses

A Horn clause has at most one positive literal. A definite clause has exactly one positive literal:

$$H \leftarrow B_1, \dots, B_n.$$

In logic form:

$$B_1 \wedge \dots \wedge B_n \Rightarrow H.$$

Facts have empty body: H . Queries ask whether a goal follows from the program.

Herbrand universe, base, and interpretations

For a first-order language:

- **Herbrand universe:** all ground terms constructible from constants and function symbols.
- **Herbrand base:** all ground atoms over predicate symbols and Herbrand universe.
- **Herbrand interpretation:** subset of the Herbrand base; atoms in it are true.

For definite programs, there is a unique least Herbrand model, obtainable as the least fixed point of the immediate consequence operator T_P .

Minimal model semantics

For definite logic programs, the declarative meaning is the least model. Operationally, SLD resolution computes answers soundly and completely with respect to this model, under fair search conditions.

SLD resolution

SLD = Selective Linear Definite-clause resolution.

Given goal:

$$\leftarrow G_1, \dots, G_k,$$

select atom G_i , unify it with the head H of some program clause

$$H \leftarrow B_1, \dots, B_m,$$

and replace G_i by $B_1, \dots, B_m$ under the most general unifier (MGU).

Prolog uses depth-first, left-to-right SLD search.

Exam tip

Prolog's operational behavior matters. The logical program may be declaratively correct, but depth-first left-to-right search can still loop or miss answers if predicates are ordered badly.

Example

Program: `parent(alice,bob). parent(bob,cara). ancestor(X,Y) :- parent(X,Y). ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).` Query `ancestor(alice,cara)` first tries direct parent and fails, then finds `Z=bob`, then proves `ancestor(bob,cara)` by the direct parent rule.

Negation as failure

Prolog's `not G` or `\\+ G` succeeds if Prolog fails to prove G .

This is not classical negation. It is based on the closed-world assumption: what cannot be proved is treated as false.

It is sound in well-behaved cases, especially for ground goals under stratified programs. With variables, it can be unintuitive:

`\\+ p(X)`

means “Prolog cannot prove any $p(X)$ ”, not “find X such that $p(X)$ is false”.

SLDNF resolution

SLDNF extends SLD with negation as failure. For negative literal $\neg A$:

- if A finitely fails, $\neg A$ succeeds,
- if A succeeds, $\neg A$ fails,
- if A loops, behavior may be non-terminating.

Cut

Prolog cut `!` commits to choices made since entering the current predicate.

Uses:

- improve efficiency by pruning alternatives,
- encode if-then-else,
- enforce determinism.

Green cut does not change declarative meaning; red cut changes meaning and must be used carefully.

3. First-Order Logic Normal Forms and Resolution

FOL normal-form pipeline

To prepare a FOL formula for resolution:

1. Eliminate $\Rightarrow, \Leftrightarrow$.
2. Move negations inward to NNF.
3. Standardize variables apart.
4. Move quantifiers to front: prenex normal form.
5. Skolemize existential quantifiers.
6. Drop universal quantifiers.
7. Convert matrix to CNF.
8. Split conjunction into clauses.

Skolemization preserves satisfiability, not equivalence. Example:

$$\forall x \exists y P(x, y)$$

becomes

$$\forall x P(x, f(x)).$$

Simple explanation

Skolemization replaces “there exists something” by a symbolic witness. If the witness depends on a universal variable, use a Skolem function like $f(x)$; if it does not depend on anything, use a Skolem constant.

Example

$\forall x \exists y \text{ Loves}(x, y)$ becomes $\forall x \text{ Loves}(x, f(x))$. The function f means “the thing loved by x .” By contrast, $\exists y \forall x \text{ Loves}(x, y)$ becomes $\forall x \text{ Loves}(x, c)$ because the witness does not depend on x .

Unification

FOL resolution needs unification because literals may contain variables and terms.

A substitution θ maps variables to terms. It unifies terms s, t if $s\theta = t\theta$.

The most general unifier (MGU) is a unifier from which all other unifiers can be obtained by further substitution.

Occurs check prevents invalid substitutions such as $x = f(x)$.

FOL resolution rule

Given clauses:

$$(A \vee L), \quad (B \vee \neg K)$$

if L and K have MGU θ , derive

$$(A \vee B)\theta.$$

Resolution is refutation-complete for first-order logic: if a set of clauses is unsatisfiable, saturation by resolution can derive $\square$ under a fair strategy.

Subsumption

Clause C subsumes clause D if there is substitution θ such that

$$C\theta \subseteq D.$$

Then D is redundant: C is at least as general. Removing subsumed clauses helps control saturation.

Example:

$$P(x) \text{ subsumes } P(a) \vee Q(b).$$

Saturation procedure

Saturation repeatedly adds logical consequences until:

- the empty clause is derived: unsatisfiable,
- no new non-redundant clauses can be generated: saturated set.

Generic loop:

1. maintain active and passive clause sets,
2. select a passive clause,
3. generate inferences with active clauses,
4. simplify by subsumption/rewriting,
5. add non-redundant results.

Saturation may not terminate for satisfiable first-order problems.

4. Equality in First-Order Logic

Axiomatic equality

One approach adds equality axioms:

- reflexivity: $x = x$,
- symmetry: $x = y \Rightarrow y = x$,
- transitivity: $x = y \wedge y = z \Rightarrow x = z$,
- congruence for functions:

$$x_i = y_i \ \forall i \Rightarrow f(x_1, \dots, x_n) = f(y_1, \dots, y_n),$$

- substitutivity for predicates.

Problem: generates many clauses and is inefficient.

Paramodulation and superposition

Instead of axiomatizing equality fully, extend resolution with equality inference.

Paramodulation idea:

from equality $s = t$ and a clause containing subterm s' unifiable with s , replace that subterm by t under the unifier.

Superposition is a refined, ordered version used in modern theorem provers. It combines:

- resolution,
- equality reasoning,
- term ordering,
- simplification by rewriting.

This is much more efficient than explicit equality axioms.

5. Searching for Models in Generic Domains

Grounding

A first-order problem over a finite domain can be grounded:

1. choose finite domain objects,
2. instantiate variables by all domain objects,
3. replace each ground atom by a propositional variable,
4. encode constraints as SAT.

Example:

$$\forall x P(x) \vee Q(x)$$

over domain $\{a, b\}$ becomes

$$(P_a \vee Q_a) \wedge (P_b \vee Q_b).$$

Propositional encoding

Common constraints:

- at least one: $x_1 \vee \dots \vee x_n$,
- at most one: $\neg x_i \vee \neg x_j$ for all pairs,
- exactly one: at least one plus at most one,
- implication: $a \Rightarrow b$ becomes $\neg a \vee b$.

Model search becomes SAT solving. This is used in planning, scheduling, puzzles, bounded verification, and finite model finding.

Grounding challenges

Grounding can explode combinatorially. If a predicate has arity k and domain size n , it has n^k ground atoms.

Mitigation:

- typed domains,
- symmetry breaking,
- constraint propagation before grounding,
- lazy grounding,
- compact cardinality encodings,
- incremental SAT.

Exam Checklist

- Convert formulas to NNF/CNF and explain Tseitin transformation.
- State resolution, unit propagation, DPLL, CDCL, and clause learning.
- Explain lazy SMT.
- Define Horn/definite clauses, Herbrand interpretations, and minimal model semantics.
- Explain SLD, SLDNF, negation as failure, and cut.
- Convert FOL to clause form and explain Skolemization.
- Explain unification, MGU, subsumption, and saturation.
- Compare equality axioms with paramodulation/superposition.
- Explain grounding and SAT encoding for model search.